# Uncertainty-Guided Test-Time Alignment for Large Language Models

## UGP Project Report

**Mentor : Prof. Sayak Ray Chowdhury**


T. Pawan Chanukya Reddy
*Roll Number: 221120*
*Computer Science and Engineering*
*Indian Institute of Technology Kanpur*
venkatasai22@iitk.ac.in

Venkatesh Akula
*Roll Number: 220109*
*Computer Science and Engineering*
*Indian Institute of Technology Kanpur*
akulav22@iitk.ac.in


## 1 Introduction

Large Language Models (LLMs) are capable of generating multiple diverse responses for the same prompt, but these responses often vary significantly in usefulness, correctness, and alignment with human preferences. A natural test-time strategy is therefore to sample several candidate outputs and choose the best among them. This idea is commonly implemented through *Best-of-N* (BoN) sampling. Given a prompt $x$, the model generates $N$ candidate responses $\{y_1, \ldots, y_N\}$ from a base policy $\pi_0(y|x)$. Each candidate is then evaluated by a learned *Reward Model* (RM), which assigns a scalar preference score $r(x, y_i)$. The final output is selected as

$$y^* = \arg\max_i r(x, y_i).$$

BoN is attractive because it improves output quality at inference time without modifying the underlying language model.

To understand why such test-time methods are useful, it is helpful to contrast them with training-time alignment methods such as Reinforcement Learning from Human Feedback (RLHF). In RLHF, a reward model is first trained from human preference comparisons, and then the language model is optimized to maximize reward while staying close to the original policy. A common objective is

$$\max_{\pi} \ \mathbb{E}_{y\sim\pi(\cdot|x)}[r(x,y)] - \beta\,\mathrm{KL}(\pi(\cdot|x) \,\|\, \pi_0(\cdot|x)),$$

where $\beta$ controls deviation from the base model. The optimal solution has the exponential-tilted form

$$\pi^*(y|x) \propto \pi_0(y|x) \exp\left(\frac{r(x,y)}{\beta}\right).$$

Although effective, RLHF requires expensive retraining and repeated optimization over large models.

BoN offers a lightweight alternative to RLHF. Instead of retraining the model, it uses the reward model only during inference to rank sampled candidates. Theoretically, as the number of samples $N$ increases, selecting the highest-scoring response increasingly approximates drawing from a policy that concentrates probability mass on high-reward outputs, making BoN a practical surrogate for aligned generation. This makes it especially appealing when compute or data constraints make full RLHF infeasible.

However, in practice, increasing $N$ does not always improve performance. Reward models are imperfect estimators of human preferences, trained on limited and noisy supervision. Their scores may be miscalibrated on difficult or out-of-distribution examples. As more candidates are sampled, the chance of encountering responses that exploit weaknesses of the reward model also increases. Consequently, BoN may select outputs that receive high reward scores despite being factually incorrect, unsafe, or low quality. This phenomenon is known as ***reward hacking***, and it motivates the need for uncertainty-aware test-time alignment methods.

### *1.1 Problem Statement*

Existing test-time alignment methods largely assume that reward model scores are fully reliable indicators of response quality. In practice, however, reward models are only approximate proxies for human preferences and are subject to noise, limited supervision, and calibration errors. As the number of sampled candidates increases, standard Best-of-$N$ methods may increasingly favor responses that exploit these inaccuracies rather than genuinely better outputs. Consequently, larger search budgets do not necessarily translate to higher-quality answers. This project addresses the need for more robust test-time alignment by incorporating not only the predicted reward score, but also the model's uncertainty or confidence in that score during candidate selection.

## 2 Constructing Uncertainty-Aware Reward Models

### 2.1 Ensemble-Based Reward Estimation

To make test-time alignment uncertainty-aware, the scoring system must provide not only an estimated reward, but also a measure of confidence in that estimate. A single reward model typically outputs only one scalar score and may be sensitive to architecture bias, training noise, or domain shift. To overcome this limitation, we use an ensemble of reward models and interpret disagreement across models as an uncertainty signal. This allows candidate responses to be evaluated using both predicted quality and reliability.

Ensemble methods are widely used for uncertainty estimation because they capture epistemic uncertainty arising from model assumptions and limited training data. If multiple independently trained models strongly agree on a candidate, the score can be treated as more reliable. Conversely, large disagreement indicates that the candidate lies in a region where preference judgments are unstable or uncertain.

### 2.2 Mean and Variance from Multiple Reward Models

Let $K$ reward models evaluate candidate response $y_i$, producing scores

$$r_{1,i}, r_{2,i}, \ldots, r_{K,i}.$$

We compute the ensemble mean and variance as

$$\mu_i = \frac{1}{K}\sum_{k=1}^{K} r_{k,i}, \qquad \sigma_i^2 = \frac{1}{K}\sum_{k=1}^{K}(r_{k,i} - \mu_i)^2.$$

Here, $\mu_i$ represents the consensus estimate of response quality, while $\sigma_i^2$ captures disagreement among models. A low variance indicates consistent judgments across reward models, whereas a high variance suggests that the score should be treated with caution. These two quantities form the basis of all uncertainty-aware selection rules used later in the project.

### 2.3 Reward Models Used in Experiments

For our experiments, we use three publicly available pretrained preference models with different architectures and training characteristics:

- **OASST-RM** (OpenAssistant DeBERTa Reward Model),
- **LLAMA-RM** (Llama-based preference model),
- **ARMO-RM** (ArmoRM).

These models were originally trained on pairwise preference data, where a preferred response is ranked above a rejected one for the same prompt. We do not retrain these reward models; instead, we use them directly as diverse evaluators during inference. Architectural diversity helps produce more informative disagreement signals than an ensemble of nearly identical models.

### 2.4 Need for Score Normalization

Since the reward models are independently trained, their raw scores are not calibrated to a common numerical scale. One model may output values in a narrow range, while another may produce larger magnitudes or systematic offsets. Directly averaging such scores can distort the ensemble mean and variance, making uncertainty estimates less meaningful.

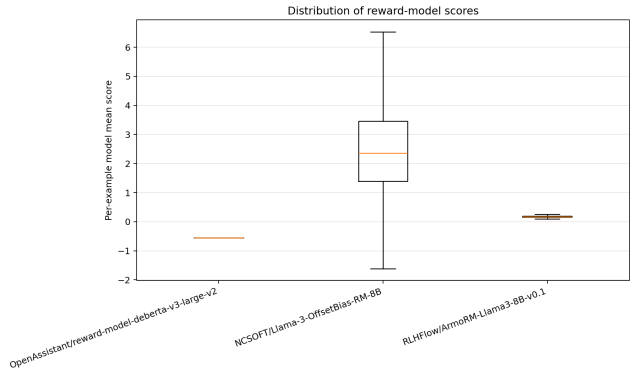



**Figure 1.** Distribution of raw scores from the three reward models. The score ranges differ significantly across models.

To address this, score normalization is required before combining outputs across models.

### 2.5 Robust Normalization Strategy

After evaluating multiple normalization methods, the most effective approach in our experiments was robust-$z$ normalization followed by hyperbolic tangent bounding. For reward model $k$ and candidate $i$, we compute

$$z_{k,i} = \frac{r_{k,i} - \text{median}(r_{k,*})}{1.4826 \cdot \text{MAD}(r_{k,*}) + \epsilon}, \qquad \widetilde{z}_{k,i} = \tanh(z_{k,i}),$$

where MAD denotes the median absolute deviation.

The robust-$z$ step removes scale and offset differences while remaining resistant to outliers. The tanh transformation then bounds extreme values to $[-1, 1]$, preventing a single model from dominating variance-sensitive selection rules. Final ensemble statistics $\mu_i$ and $\sigma_i^2$ are computed using these normalized scores. This normalization produced the most consistent improvement in downstream answer selection accuracy.

### 2.6 Normalization Pipeline

Normalization is applied **after generating $N$ candidates and scoring them**, and **before** ensemble aggregation.

For one prompt $x$:

1. Generate $N$ candidates $\{y_1, \ldots, y_N\}$.
2. Each reward model $k$ scores every candidate, producing a vector $(r_{k,1}, \ldots, r_{k,N})$.

3. Normalize **within each model** over these $N$ values.
4. Aggregate across models to get $(\mu_i, \sigma_i^2)$ per candidate.
5. Apply selection policy.

**Method A: Min-Max normalization (ablation).** For model $k$ and candidate $i$:

$$r'_{k,i} = \frac{r_{k,i} - \min_j r_{k,j}}{\max_j r_{k,j} - \min_j r_{k,j} + \epsilon_{\text{norm}}}.$$

This maps scores to $[0, 1]$. We found this sensitive to outliers and range compression.

**Method B: Robust z-score + tanh (main pipeline).** For model $k$:

$$m_k = \text{median}_j(r_{k,j}), \qquad \text{MAD}_k = \text{median}_j\left(|r_{k,j} - m_k|\right),$$

$$z_{k,i} = \frac{r_{k,i} - m_k}{1.4826 \cdot \text{MAD}_k + \epsilon_{\text{norm}}}, \qquad \tilde{z}_{k,i} = \tanh(z_{k,i}) \in [-1, 1].$$

We use $s_{k,i} = \tilde{z}_{k,i}$ for aggregation.

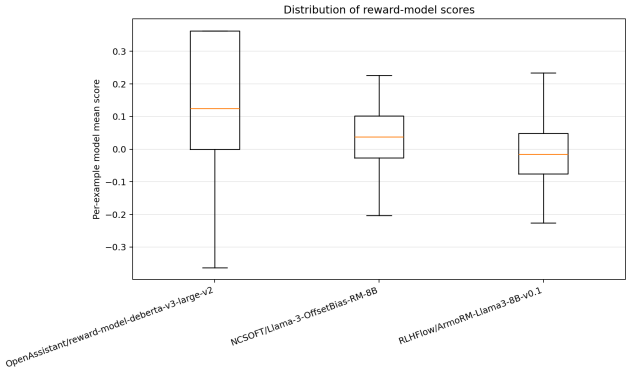



**Figure 2.** Scale recovery after robust normalization.

## 3 Uncertainty-Aware Test-Time Alignment Methods

Traditional Best-of-$N$ methods rely only on reward scores when selecting responses, implicitly assuming that these scores are fully reliable. In our framework, each candidate response is associated with both an estimated reward $\mu_i$ and an uncertainty measure $\sigma_i^2$. Hence, a stronger selection strategy should use both quantities. Intuitively, desirable candidates should have high mean reward, indicating likely quality, and low uncertainty, indicating confidence in that estimate.

We study two major classes of test-time alignment methods: *parallel* and *sequential*. In parallel methods, all $N$ candidates are generated independently and then ranked jointly. In sequential methods, candidates are generated one at a time, and accepted candidates can influence future generations. We also study *SoftBoN* variants, where responses are sampled probabilistically through a softmax distribution instead of deterministic arg max selection.

### 3.1 Parallel Best-of-$N$ Methods

Let $N$ candidate responses be scored as $(\mu_i, \sigma_i^2)$ for $i = 1, \ldots, N$.

**Vanilla BoN.** The standard baseline selects the candidate with maximum predicted reward:

$$y^* = \arg\max_i \mu_i.$$

This assumes the reward score alone is sufficient.

**Sharpe BoN.**

$$y^* = \arg\max_i \frac{\mu_i}{\sigma_i + \epsilon}.$$

This acts as a signal-to-noise ratio, favoring candidates with high reward and low uncertainty.

**Linear BoN.**

$$y^* = \arg\max_i (\mu_i - c\sigma_i),$$

where $c > 0$ penalizes uncertain candidates. This directly balances reward and reliability.

**SoftBoN Variants.** Instead of deterministic selection, SoftBoN samples according to:

$$p(i) \propto \exp(u_i / T),$$

where $u_i$ is a utility score and $T$ is the temperature. Lower $T$ behaves closer to argmax, while larger $T$ increases exploration.

| Method | Selection Rule | Key Idea |
|---|---|---|
| Vanilla BoN | $\arg\max_i \mu_i$ | Highest mean reward |
| Sharpe BoN | $\arg\max_i \mu_i/(\sigma_i + \epsilon)$ | Reward per uncertainty |
| Linear BoN | $\arg\max_i(\mu_i - c\sigma_i)$ | Penalize uncertainty |
| SoftBoN | $p(i) \propto \exp(\mu_i/T)$ | Probabilistic ranking |
| SoftBoN-Sharpe | $p(i) \propto \exp\left(\frac{\mu_i}{\sigma_i+\epsilon}/T\right)$ | Soft + confidence |
| SoftBoN-Linear | $p(i) \propto \exp((\mu_i - c\sigma_i)/T)$ | Soft + risk penalty |

**Table 1**
Parallel uncertainty-aware Best-of-$N$ methods.

### 3.2 Sequential Best-of-$N$ Methods

Sequential BoN methods generate candidates one at a time rather than all at once. After each generation, a candidate is evaluated and accepted into the running context only if it passes a threshold. Accepted candidates can then influence future generations, allowing iterative refinement.

Let $a_i$ denote the $i$-th generated candidate.

**Vanilla SeqBoN.** Accept candidate $a_i$ if

$$\mu_i \geq \gamma.$$

**Sharpe SeqBoN.** Accept if

$$\frac{\mu_i}{\sigma_i + \epsilon} \geq \gamma.$$

**Linear SeqBoN.** Accept if

$$\mu_i - c\sigma_i \geq \gamma.$$

These methods use uncertainty not only during final ranking, but also during generation by deciding which candidates are strong enough to influence future outputs.

**Algorithm 1** Sequential Best-of-$N$ (SeqBoN)

```
Initialize history ← x
for i = 1 to N do
    Generate candidate a_i ~ π(· | history)
    Compute (μ_i, σ_i^2)
    if score(a_i) ≥ γ then
        history ← history + a_i
    end if
end for
Return best candidate among all generated responses
```

where score($a_i$) is chosen as $\mu_i$, $\mu_i/(\sigma_i + \epsilon)$, or $\mu_i - c\sigma_i$ depending on the variant used.

| Variant | Acceptance Criterion | Purpose |
|---|---|---|
| Vanilla SeqBoN | $\mu_i \geq \gamma$ | Reward thresholding |
| Sharpe SeqBoN | $\mu_i/(\sigma_i + \epsilon) \geq \gamma$ | Confidence-aware filtering |
| Linear SeqBoN | $\mu_i - c\sigma_i \geq \gamma$ | Penalize risky candidates |

**Table 2**
Sequential uncertainty-aware Best-of-$N$ variants.

## 4 Experimental Setup

### 4.1 Models and Datasets

- Generator: `Qwen/Qwen2.5-7B-Instruct`
- Reward-model ensemble: OASST-RM, LLAMA-RM, ARMO-RM
- GSM8K test set: 1,319 examples
- MATH500 test set: 500 examples

### 4.2 Evaluation Protocol

To keep comparisons fair, methods share the same per-prompt candidate pool and reward signals; only the selection utility changes. This avoids confounding gains from different generations.

### 4.3 Core Hyperparameters

**Table 3**
Primary inference hyperparameters used in major runs

| Parameter | Value |
|---|---|
| BoN candidates ($N$) | 8 |
| Sequential rounds ($n$) | 5 |
| History budget | 2 |
| Generation temperature | 0.7 |
| Max new tokens | 512 |
| Sharpe epsilon | $10^{-6}$ to $10^{-8}$ (run dependent) |
| Linear penalty $c$ | 1.0 |
| Normalization mode | robust_zscore (main), minmax (ablation), none (baseline) |
| Tanh bound | enabled in robust-z bounded runs |
| Top-$k$ for disagreement | 3 |
| Hybrid threshold $D_{thr}$ | 0.290468 (fixed-threshold evaluation) |

## 5 Results and Interpretation

### 5.1 Before Normalization: Baseline Behavior

Before robust normalization, methods were competitive but unstable across datasets. GSM8K had a narrow range (87.8% to 89.1%), while MATH500 showed larger gaps and stronger sensitivity to utility choice.

**Table 4**
Before normalization on GSM8K (1,319). Methods are close, but ranking differences are small and noisy.

| Method | Acc. | Avg $\mu$ | Avg Var | RM Calls | Time |
|---|---|---|---|---|---|
| vanilla_bon | 88.5% | 1.66 | 9.07 | 1.0 | 27.1s |
| bon_sharpe | 88.7% | 1.65 | 9.05 | 1.0 | 27.1s |
| bon_linear | 87.8% | 0.27 | 0.75 | 1.0 | 27.1s |
| soft_bon | 88.3% | 1.15 | 6.08 | 1.0 | 27.1s |
| softbon_sharpe | 89.0% | 0.78 | 3.40 | 1.0 | 27.1s |
| softbon_linear | 89.1% | 0.60 | 2.38 | 1.0 | 27.1s |

**Table 5**
Before normalization on MATH500 (500): sharper method differences indicate calibration sensitivity.

| Method | Accuracy | Avg $\mu$ | Avg variance | RM calls | Latency |
|---|---|---|---|---|---|
| vanilla_bon | 69.6% | 2.0486 | 10.195375 | 1.0 | 47.54s |
| bon_sharpe | 70.0% | 2.0337 | 10.037772 | 1.0 | 47.54s |
| bon_linear | 66.4% | 0.6213 | 1.961176 | 1.0 | 47.54s |
| soft_bon | 64.6% | 1.5330 | 7.070341 | 1.0 | 47.54s |
| softbon_sharpe | 66.2% | 1.3385 | 5.941614 | 1.0 | 47.54s |
| softbon_linear | 66.2% | 1.0948 | 4.638202 | 1.0 | 47.54s |

**Interpretation**: In this stage, we can see the methods are producing almost same kind of accuracy results. Even when accuracy looked acceptable, utility rankings were not consistent enough to trust globally and no clear pattern is visible. That is what motivated us to try the normalization-focused redesign.

### 5.2 After Robust Normalization

Once robust normalization was enabled, method behavior became more stable and top-line accuracy improved on both the datasets.

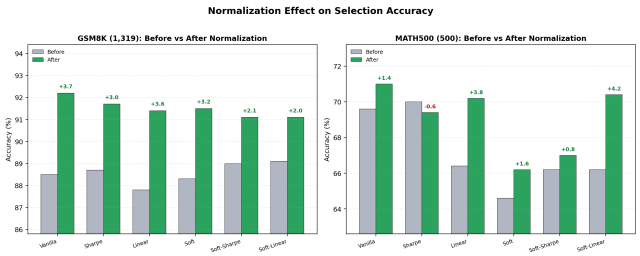



**Figure 3.** Before-vs-after normalization comparison across methods for GSM8K and MATH500.

**Table 6**
After robust normalization on GSM8K (1,319): best method reaches 92.2%, improving over pre-normalization best.

| Method | Accuracy | Avg $\mu$ | Avg variance | RM calls |
|---|---|---|---|---|
| vanilla_bon | 92.2% | 0.5162 | 0.163850 | 1.0 |
| soft_bon | 91.5% | 0.1461 | 0.236250 | 1.0 |
| bon_sharpe | 91.7% | 0.4853 | 0.108378 | 1.0 |
| bon_linear | 91.4% | 0.4894 | 0.104562 | 1.0 |
| softbon_sharpe | 91.1% | 0.3210 | 0.188859 | 1.0 |
| softbon_linear | 91.1% | 0.1383 | 0.201680 | 1.0 |

**Table 7**
After robust normalization on MATH500 (500). Accuracy and variance profile both improve relative to raw runs.

| Method | Acc. | Avg $\mu$ | Avg Var | RM Calls | Time |
|---|---|---|---|---|---|
| vanilla_bon | 71.0% | 0.55 | 0.138 | 1.0 | 48.0s |
| soft_bon | 66.2% | 0.15 | 0.211 | 1.0 | 48.0s |
| bon_sharpe | 69.4% | 0.52 | 0.104 | 1.0 | 48.0s |
| bon_linear | 70.2% | 0.53 | 0.102 | 1.0 | 48.0s |
| softbon_sharpe | 67.0% | 0.33 | 0.175 | 1.0 | 48.0s |
| softbon_linear | 70.4% | 0.16 | 0.177 | 1.0 | 48.0s |

**Interpretation**: the biggest practical gain is not only higher absolute accuracy but improved consistency across all the methods. For example, GSM8K best moves from 89.1% to 92.2%, and results on MATH500's dataset also has improved across all methods. This supports our claim that normalization quality is foundational for fair ensemble selection.

### 5.3 Min-Max Ablation

We kept Min-Max as a sanity-check ablation because it is commonly used and easy to implement.

**Table 8**
Min-Max normalization ablation on GSM8K (1,319): strong drop versus robust normalization.

| Method | Accuracy | Avg $\mu$ | Avg variance |
|---|---|---|---|
| vanilla_bon | 80.7% | 0.6492 | 0.0871 |
| bon_sharpe | 80.5% | 0.6294 | 0.0732 |
| bon_linear | 80.8% | 0.6397 | 0.0773 |
| soft_bon | 76.9% | 0.4244 | 0.1029 |
| softbon_sharpe | 78.6% | 0.5606 | 0.0721 |
| softbon_linear | 77.0% | 0.4217 | 0.0957 |

**Interpretation**: this is one of the clearest negative results in the project. Min-Max drops vanilla BoN to 80.7% on GSM8K, more than 11 points below robust-normalized vanilla BoN (92.2%). Min-max is sensitive to outliers and collapses ranking information. This was our biggest lesson that a wrong preprocessing choice completely destroys performance.

### 5.4 Sequential Filtering Results

Sequential filtering gave the strongest controlled gains in robust-mode runs. It increased latency, but the accuracy gain was often large enough to justify the extra compute. Also, it can clearly seen that value of the threshold gamma largely influences the results.

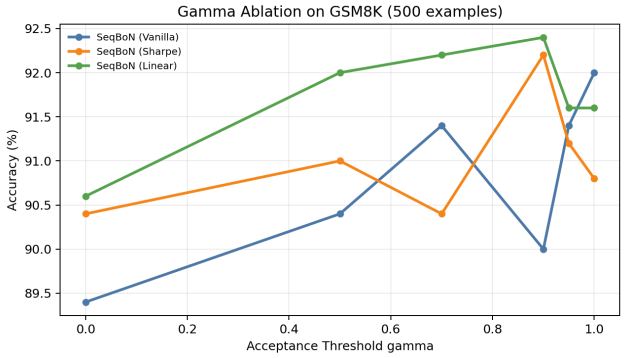



**Figure 4.** Sensitivity of sequential methods to acceptance threshold $\gamma$ on GSM8K-500.

**Table 9**
Robust-mode GSM8K (1,319): sequential variants outperform parallel baselines with higher RM-call cost.

| Method | Accuracy | Avg $\mu$ | RM calls | Latency |
|---|---|---|---|---|
| vanilla_bon | 87.4% | 5.5747 | 1.0 | 61.27s |
| risk_adjusted_bon | 87.4% | 5.5747 | 1.0 | 61.27s |
| sharpe_ratio_bon | 87.0% | 5.5474 | 1.0 | 61.27s |
| vanilla_seqbon | 91.9% | 5.5680 | 5.0 | 81.45s |
| variance_aware_seqbon | 90.7% | 5.5675 | 5.0 | 81.99s |

**Table 10**
Robust-mode MATH500 (500): Sharpe-sequential reaches the best score (72.2%).

| Method | Accuracy | Avg $\mu$ | RM calls | Latency |
|---|---|---|---|---|
| vanilla_bon | 66.0% | 5.8960 | 1.0 | 39.83s |
| risk_adjusted_bon | 66.0% | 5.8959 | 1.0 | 39.83s |
| sharpe_ratio_bon | 66.6% | 5.8198 | 1.0 | 39.83s |
| vanilla_seqbon | 70.0% | 5.8914 | 5.0 | 78.05s |
| Sharpe_seqbon | 72.2% | 5.8402 | 5.0 | 77.91s |
| Lin_seqBon | 68.2% | 5.8821 | 5.0 | 78.00s |

**Interpretation**: On GSM8K, vanilla sequential improves from 87.4% to 91.9% over parallel vanilla. On MATH500, Sharpe sequential improves from 66.6% (parallel Sharpe) to 72.2%. These are large gains for an inference-only intervention. Figure 4 also shows why tuning $\gamma$ matters: too low admits weak steps; too high blocks useful corrections.

### *5.5 Reward-Hacking Proxy and Hybrid Routing*

To understand behavior beyond top-line accuracy, we tracked a reward-hacking proxy based on selected mean and selected variance.

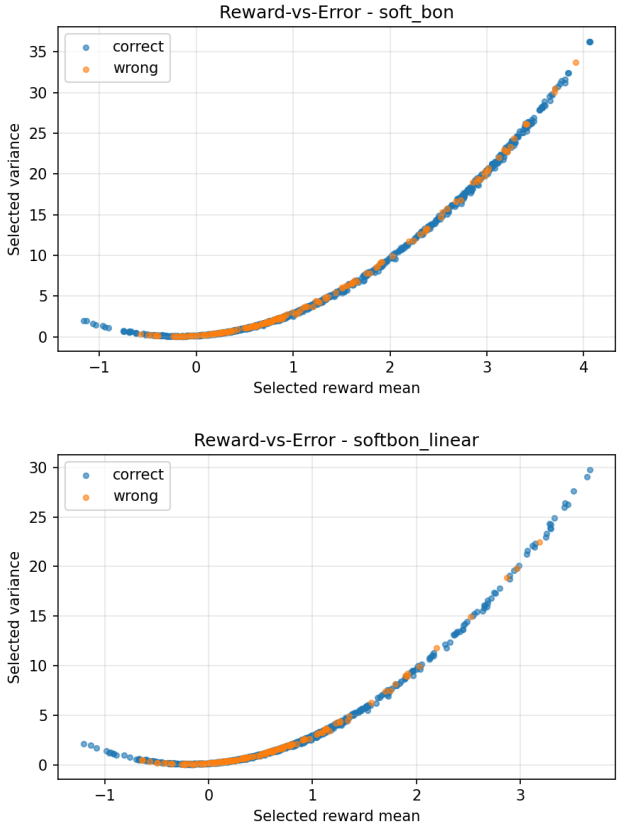



**Figure 5.** Reward-hacking proxy scatter: SoftBoN vs SoftBoN-Linear.

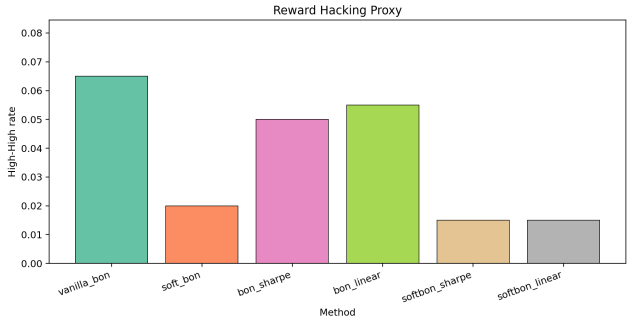



**Figure 6.** Rate of high-reward/high-variance selections across methods.

Here high reward/high variance means candidates who has reward scores in Top 25% range of reward/variance.

**Table 11**
Disagreement-gated hybrid on GSM8K (1,319): near-best accuracy with improved interpretability of routing decisions.

| Policy | Accuracy | Avg reward |
|---|---|---|
| vanilla_bon | 92.19% | 0.5162 |
| bon_sharpe | 91.66% | 0.4853 |
| hybrid(vanilla, sharpe) | 91.74% | 0.5060 |

**Interpretation**: uncertainty-aware utilities reduce concentration in the high-mean/high-variance corner (Figures 5 and 6). The hybrid policy is not always top-1 in accuracy, but it is close while preserving a transparent decision rule based on hybrid disagreement equation.

### *5.6 Reward Hacking with Increasing N*

To study reward hacking under larger search budgets, we compare the accuracy of multiple test-time alignment methods as the number of sampled candidates $N$ increases. The experiment contrasts vanilla BoN and Inference-Time Pessimism with uncertainty-aware approaches such as Sharpe BoN, Sharpe SoftBoN.

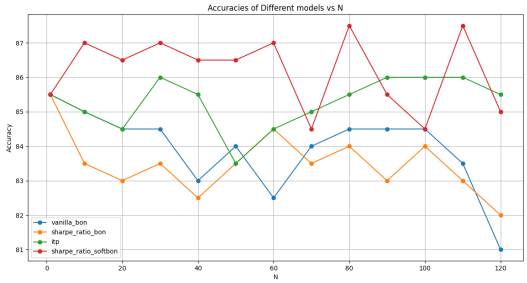



**Figure 7.** Accuracy trends with increasing $N$ across different alignment methods.

### 5.7 Final Benchmark

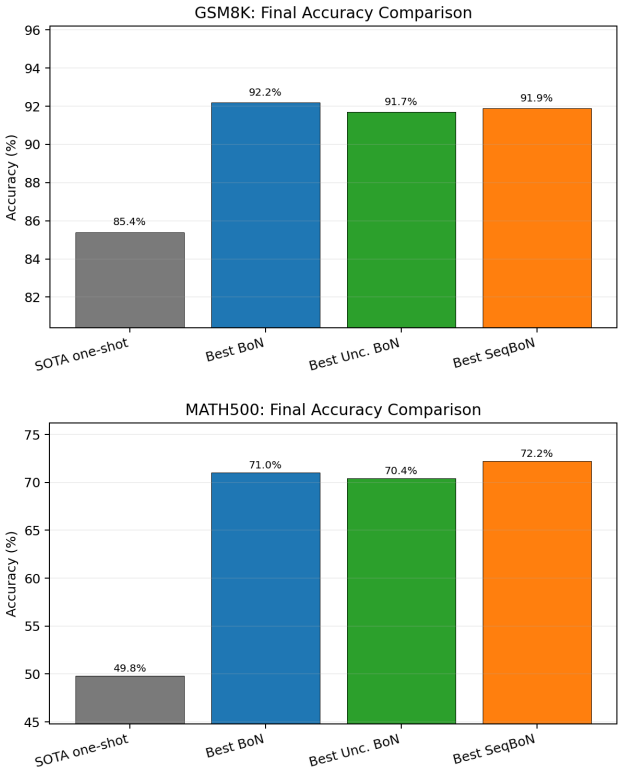



**Figure 8.** Final benchmark comparison against one-shot baseline on GSM8K and MATH500.

To put things in perspective, we also compare our results with the one-shot SOTA accuracies of Qwen-2.5 model on GSM8K and MATH500 datasets. The benchmark summary confirms that all the final inference stacks improves accuracy over one-shot baseline behavior on both datasets as expected.

### 5.8 Takeaway

The results support a simple but important point: in test-time alignment, score calibration and uncertainty handling are as important as search depth. Early in the project, we assumed utility choice (vanilla vs Sharpe vs linear) would dominate outcomes. After full ablations, we found that normalization quality often determined whether those utility comparisons were meaningful at all.

A second lesson is that sequential filtering is useful when reasoning needs correction across attempts. The gains on robust-mode sequential runs were not marginal. At the same time, the cost is high: RM calls increase from 1 to 5 and latency rises accordingly. For practical deployment, this means policy choice should depend on latency budget.

A third takeaway is about reliability signals. Disagreement variance is not a perfect uncertainty decomposition, but it was operationally useful in two places: (i) utility-level risk adjustment and (ii) hybrid routing. This is why we keep disagreement features in the final pipeline even when they do not always maximize raw accuracy.

## 6 Limitations and Future Work

### 6.1 Limitations

Although the proposed uncertainty-aware methods produced encouraging improvements, several limitations remain. First, available compute resources constrained extensive large-$N$ sweeps and repeated multi-seed evaluations for every configuration, limiting the statistical depth of comparisons. Second, some methods depend on hyperparameters such as the acceptance threshold $\gamma$ and uncertainty penalty coefficient $c$, which still require empirical tuning for best performance. Third, while ensemble disagreement variance provides a practical uncertainty signal, it is not a complete decomposition of uncertainty and may fail to fully capture all forms of distribution shift or reward model error.

### 6.2 Future Work

There are several promising directions for extending this work. One direction is variance-trend control during multi-step reasoning, where generation can be stopped, rerouted, or re-sampled when uncertainty rises consistently across steps. Another direction is per-prompt adaptive thresholding for sequential methods, allowing acceptance criteria to depend on prompt difficulty. Future systems could also combine process-level reward models with outcome-level reward models to estimate confidence at intermediate reasoning steps as well as final answers. Finally, larger Best-of-$N$ sweeps with confidence intervals and evaluation on more challenging benchmarks such as GPQA would provide a stronger understanding of the robustness and scalability of uncertainty-aware test-time alignment.

## 7 Acknowledgement

We would like to express our sincere gratitude to Prof. Sayak Ray Chowdhury for his constant guidance, encouragement, and support throughout the course of this project. His insights and mentorship were invaluable in shaping the direction of this work.

We would also like to thank the Department of Computer Science and Engineering for providing the computational resources, including GPU support, required to conduct our experiments.

The code of the proposed methods and experimental pipeline is available in the GitHub repository here.

## References

[1] Audrey Huang, Adam Block, Qinghua Liu, Nan Jiang, Akshay Krishnamurthy, and Dylan J. Foster. *Is Best-of-N the Best of Them? Coverage, Scaling, and Optimality in Inference-Time Alignment*. arXiv preprint, 2025.

[2] Debangshu Banerjee and Aditya Gopalan. *Towards Reliable Alignment: Uncertainty-aware RLHF*. arXiv preprint, 2024.

[3] Yue Yu, Qiwei Di, Quanquan Gu, and Dongruo Zhou. *On the Limits of Test-Time Compute: Sequential Reward Filtering for Better Inference*. arXiv preprint, 2025.

[4] Lin Gui, Cristina Garbacea, and Victor Veitch. *BoNBoN Alignment for Large Language Models and the Sweetness of Best-of-N Sampling*. arXiv preprint, 2024.

[5] Soumya Suvra Ghosal, Souradip Chakraborty, Avinash Reddy, Yifu Lu, Mengdi Wang, Dinesh Manocha, Furong Huang, Mohammad Ghavamzadeh, and Amrit Singh Bedi. *Does Thinking More Always Help? Mirage of Test-Time Scaling in Reasoning Models*. arXiv preprint, 2025.

[6] Xingzhou Lou, Dong Yan, Wei Shen, Yuzi Yan, Jian Xie, and Junge Zhang. *Uncertainty-aware Reward Model: Teaching Reward Models to Know What is Unknown*. arXiv preprint, 2024.

[7] Claudio Mayrink Verdun, Alex Oesterling, Himabindu Lakkaraju, Flavio P. Calmon. *Soft Best-of-n Sampling for Model Alignment* arXiv preprint, 2025.